

SISTEMAS OPERATIVOS

Algoritmos de Planificación de Procesos

Informe de Actividad Práctica

Universidad de Las Américas — UDLA

Iván Ortiz-Garcés

2025

1. Introducción

Los **algoritmos de planificación de procesos** son mecanismos fundamentales de los sistemas operativos que determinan el orden en que los procesos acceden a la CPU. Su objetivo principal es optimizar el uso del procesador, garantizando equidad entre procesos y minimizando tiempos de espera y retorno. El presente informe analiza y resuelve los cuatro algoritmos estudiados en clase: **FIFO**, **SJF**, **Prioridad** y **Round Robin**, utilizando el mismo conjunto de procesos para facilitar la comparación.

1.1 Objetivos de un algoritmo de planificación

- **Equidad:** todos los procesos reciben tiempo de CPU de manera justa.
- **Maximizar** la utilización de la CPU y la productividad (throughput).
- **Minimizar** el tiempo de espera, el tiempo de retorno y el tiempo de respuesta.

1.2 Datos de entrada — procesos de referencia

Los cinco procesos siguientes se emplean en los cuatro algoritmos para permitir comparaciones directas:

Proceso	Ráfaga CPU (ut)	Tiempo de Llegada	Prioridad
A	3	2	2
B	1	4	3
C	3	0	1
D	4	1	3
E	2	3	4

2. Algoritmo FIFO (First-In, First-Out)

FIFO — también llamado **FCFS (First Come, First Served)** — ejecuta los procesos en el orden de llegada. Es no apropiativo: una vez que un proceso toma la CPU, la retiene hasta completarse. Su implementación es sencilla pero puede generar el *efecto convoy*, donde procesos cortos deben esperar detrás de uno largo.

Orden de ejecución (por llegada):

C llega primero (t=0), luego D (t=1), A (t=2), E (t=3), B (t=4).

C	D	A	E	B
0 ■■ 3 ■■■■■■■■ 7 ■■■■ 10 ■■ 12 ■ 13 (unidades de tiempo)				
Proceso	Inicio	Fin	T. Espera	T. Retorno
C	0	3	0-0 = 0	3
D	3	7	3-1 = 2	6
A	7	10	7-2 = 5	8

E	10	12	$10-3 = 7$	9
B	12	13	$12-4 = 8$	9
Promedio			4.4 ut	7.0 ut

Tiempo Medio de Espera (TEM):	$(0+2+5+7+8) / 5 = 4.4 \text{ ut}$
Tiempo Medio de Retorno (TRM):	$(3+6+8+9+9) / 5 = 7.0 \text{ ut}$
Tipo:	No apropiativo
Ventaja:	Simple de implementar
Desventaja:	Efecto convoy — procesos largos bloquean a los cortos

3. Algoritmo SJF (Shortest Job First)

SJF selecciona, en cada instante disponible, el proceso con la ráfaga de CPU más corta entre los que ya han llegado. En su variante **no apropiativa**, si un proceso está corriendo no se interrumpe aunque llegue uno más corto. SJF minimiza el tiempo medio de espera de manera óptima para un conjunto fijo de procesos, pero requiere conocer de antemano la duración de cada ráfaga.

Selección en $t=0$: solo C disponible → ejecuta C (3 ut).

En $t=3$ han llegado A(3), D(4), E(2) y B(1). Se elige B(1), luego E(2), A(3), D(4).



Proceso	Inicio	Fin	T. Espera	T. Retorno
C	0	3	0	3
E	3	5	$3-3 = 0$	2
B	5	6	$5-4 = 1$	2
A	6	9	$6-2 = 4$	7
D	9	13	$9-1 = 8$	12
Promedio			2.6 ut	5.2 ut

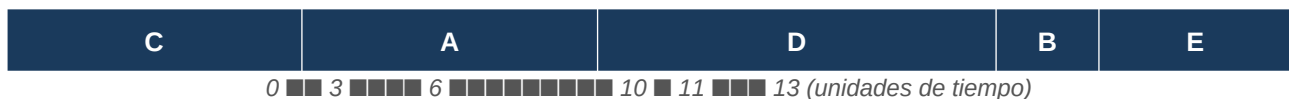
Tiempo Medio de Espera (TEM):	$(0+0+1+4+8) / 5 = 2.6 \text{ ut}$
Tiempo Medio de Retorno (TRM):	$(3+2+2+7+12) / 5 = 5.2 \text{ ut}$
Tipo:	No apropiativo
Ventaja:	Minimiza el TEM — optimo para conjuntos fijos de procesos
Desventaja:	Requiere conocer la duracion de cada rafaga; posible inanicion de procesos largos

4. Algoritmo de Prioridad

El algoritmo de **Prioridad** asigna a cada proceso un número de prioridad; el planificador elige siempre el proceso listo con mayor prioridad (número menor = mayor prioridad en este ejercicio). En su forma **no apropiativa**, el proceso en ejecución continúa hasta terminar. El principal riesgo es la *inanición* de procesos de baja prioridad, que puede mitigarse con *envejecimiento* (*aging*).

Orden por prioridad (1 = mayor prioridad):

C(p=1) → A(p=2) → D(p=3) → B(p=3, llega después) → E(p=4).



Proceso	Inicio	Fin	Prioridad	T. Espera	T. Retorno
C	0	3	1	0-0 = 0	3
A	3	6	2	3-2 = 1	4
D	6	10	3	6-1 = 5	9
B	10	11	3	10-4 = 6	7
E	11	13	4	11-3 = 8	10
Promedio				4.0 ut	6.6 ut

Tiempo Medio de Espera (TEM):	$(0+1+5+6+8) / 5 = 4.0 \text{ ut}$
Tiempo Medio de Retorno (TRM):	$(3+4+9+7+10) / 5 = 6.6 \text{ ut}$
Tipo:	No apropiativo
Ventaja:	Permite diferenciar la importancia de los procesos del sistema
Desventaja:	Inanición de procesos de baja prioridad

5. Algoritmo Round Robin (RR)

Round Robin asigna a cada proceso un *quantum* (q) fijo de tiempo de CPU. Si el proceso no termina en su quantum, es interrumpido y colocado al final de la cola de listos. Es apropiativo y garantiza equidad, siendo ideal para sistemas de tiempo compartido. Con $q = 3 \text{ ut}$ se obtiene la siguiente ejecución:

Ejecución con quantum $q = 3 \text{ ut}$:

t=0: C entra. t=3: C termina (usó 3 ut exactos); D, A y E han llegado. t=3→6: D corre 3 ut (le quedan 1 ut). t=6→9: A corre 3 ut (termina). t=9→10: E llega con 2 ut, corre 2 ut y termina en t=11. t=11→12: B (1 ut) corre y termina. t=12→13: D retoma su 1 ut restante.



Proceso	Inicio(s)	Fin	T. Espera	T. Retorno
C	0	3	0-0 = 0	3
D	3 / 12	13	(3-1)+(12-6) = 8	12
A	6	9	6-2 = 4	7
E	9	11	9-3 = 6	8
B	11	12	11-4 = 7	8
Promedio			5.0 ut	7.6 ut

Tiempo Medio de Espera (TEM):	$(0+8+4+6+7) / 5 = 5.0 \text{ ut}$
Tiempo Medio de Retorno (TRM):	$(3+12+7+8+8) / 5 = 7.6 \text{ ut}$
Tipo:	Apropiativo — quantum q = 3 ut
Ventaja:	Equitativo; adecuado para sistemas interactivos y tiempo compartido
Desventaja:	Mayor overhead por cambios de contexto; sensible al valor del quantum

6. Comparación de Algoritmos

La siguiente tabla resume los resultados obtenidos con los cinco procesos de referencia. Según el criterio del menor tiempo medio de espera, **SJF** resulta el algoritmo más eficiente para este conjunto de datos.

Algoritmo	TEM (ut)	TRM (ut)	Apropiativo	Complejidad
FIFO	4.4	7.0	No	Baja
SJF	2.6	5.2	No	Media
Prioridad	4.0	6.6	No	Media
Round Robin	5.0	7.6	Si	Media-Alta

Análisis por métrica:

- **Menor TEM** → **SJF (2.6 ut)**: minimiza los tiempos de espera al priorizar ráfagas cortas.
- **Mayor equidad** → **Round Robin**: ningún proceso espera más de q unidades entre ejecuciones.
- **Menor overhead** → **FIFO**: implementación más simple, sin necesidad de ordenar la cola.
- **Control de prioridades del sistema** → **Prioridad**: permite reservar CPU para procesos críticos.

7. Resumen y Conclusiones

El estudio de los cuatro algoritmos de planificación de procesos permite extraer las siguientes conclusiones:

- **No existe un algoritmo universal:** cada entorno requiere evaluar sus necesidades específicas (interactividad, criticidad de procesos, homogeneidad de ráfagas).
- **SJF es óptimo en TEM:** cuando se conoce la duración de las ráfagas, minimiza el tiempo promedio de espera más que cualquier otro algoritmo no apropiativo.
- **FIFO es la base conceptual:** su simplicidad lo hace ideal para entender los conceptos de planificación, aunque el efecto convoy lo penaliza en cargas heterogéneas.
- **Round Robin es la elección para tiempo compartido:** el quantum debe calibrarse: muy pequeño genera overhead excesivo; muy grande se acerca al comportamiento de FIFO.
- **Prioridad ofrece flexibilidad:** pero debe complementarse con aging para evitar la inanición indefinida de procesos de baja prioridad.
- **La concurrencia y los semáforos son complementarios:** los algoritmos de planificación gestionan el acceso a la CPU, mientras que los semáforos (Wait/Signal) sincronizan el acceso a recursos compartidos, evitando condiciones de carrera.

7.1 Recomendación para la empresa (caso ABR)

Ante un escenario empresarial donde se busca optimizar el uso de la CPU para una mezcla de procesos con distintas duraciones, se recomienda adoptar **SJF** en entornos de procesamiento por lotes donde la duración de las tareas es conocida o estimable, y **Round Robin** con un quantum ajustado para ambientes interactivos o multiusuario. Si existe jerarquía de servicios críticos, la variante **Prioridad + aging** garantiza que los procesos del sistema reciban atención inmediata sin dejar en inanición al resto.